

MiniEval：小程序合规违规自动检测与定量隐私风险评估

摘要背景

:

作为广泛应用的移动计算范式，小程序引发了对其隐私风险的高度关注。尽管既有研究已解决数据过度收集等问题，但两个关键问题仍被忽视：当无关数据请求被拒绝时功能受限，以及数据收集声明目的模糊或不合逻辑。同时我们发现，此前研究尚未提供评估小程序隐私风险的量化框架。

研究目标：

本文目标双重：设计自动化检测小程序合规违规的框架，并构建定量模型将违规行为转化为综合风险评分，进而将小程序划分为不同风险等级。

方法：

我们提出MiniEval框架，用于自动检测和定量评估隐私风险。该框架正式化了两个创新概念：间接违规（捕捉强制性功能限制）与目的合理性（评估声明目的的必要性及透明度）。为实现这些概念，MiniEval融合混合分析与大型语言模型（LLM）技术，同步检测直接与间接违规行为。其进一步运用LLM识别非理性目的，最终通过量化公式推导出综合隐私风险评分，并采用聚类方法对风险等级进行分类。

研究成果：

在350个微信小程序的基准数据集上，MiniEval展现出高识别效能：目的合理性分类F1值达90.5%，间接违规检测F1值达81.5%。 后续

大规模分析揭示：严重隐私风险既普遍存在，又高度集中于特定类别。

结论：

MiniEval 是首个为小程序生态系统整合全面合规违规检测与定量风险评估的框架。

关键词：MiniApps、大型语言模型、隐私风险评估

1. 引言

小程序作为一种变革性的移动计算范式，近年来迅速崛起。它们运行于微信、支付宝、抖音等大型宿主平台内，这种架构为用户提供了从购物、问诊到社交游戏、内容分享等广泛活动的统一体验，无需单独安装应用。然而小程序与宿主平台的深度融合引发重大隐私担忧，包括敏感用户数据的过度使用及数据收集缺乏透明度等问题，持续激化隐私风险的讨论。

随着全球对隐私保护的关注日益增强，《通用数据保护条例》(GDPR) [1]、《加州消费者隐私法案》(CCPA) [2]、《个人信息保护法》(PIPL) [3]以及《加拿大消费者隐私保护法》(CPPA)等法规相继出台。

[4]相继颁布。这些法规体现了《通用数据保护条例》(GDPR)[1]中明确界定的三大核心数据保护原则：*目的限制*、*数据最小化与透明度*。目的限制要求数据收集必须服务于特定且合法的目的。在此基础上，数据最小化原则将数据收集严格限制在实现预定目的所必需的范围内。最后，透明度原则要求目的声明必须清晰且便于用户理解。与此同时，微信等小程序平台已引入隐私政策和更严格的数据审查机制[5]。然而，在缺乏有效执法和监管监督的情况下，这些法律政策仍不足以保护用户免受重大隐私风险，凸显了加强隐私保护工具的迫切需求。

针对小程序的隐私问题研究已催生多种工具与框架。例如SPOChecker[6]可检测

在小程序中过度收集隐私数据的问题上，MiniScope[7]构建依赖图谱以检验声明与实际操作的一致性，而王世韬等人[8]则运用大型语言模型从使用场景中识别数据最小化原则的违规行为。尽管这些现有分析颇具价值，但其研究范围仅聚焦于小程序行为与其政策声明的一致性。它们忽略了另一个根本问题：声明本身的内在有效性。缺乏清晰度或必要性的用途声明本质上违背了数据保护原则。具体而言，缺乏清晰度违反透明度和用途限制原则，而缺乏必要性则违背数据最小化原则。这促使我们提出"用途合理性"概念，通过评估声明用途的清晰度和必要性来判断其合理性。与此同时，现有研究忽略了另一种同等重要的滥用形式：间接违规。此类违规并非直接过度收集个人数据，而是通过将数据与无关功能访问暗中捆绑，迫使用户披露非必要数据。这种模式通过索取非必要数据直接违反数据最小化原则，同时通过引入解锁功能的隐蔽目的，违反了目的限制与透明度原则。

我们的工作：基于这些洞见，我们提出一种新视角，通过引入"目的合理性"与"间接违规"概念，在声明维度与行为维度双重层面实现数据保护原则的可操作化。为此，我们设计并实现了自动化隐私审计系统框架MiniEval。首先，该框架运用基于大型语言模型的推理模块，对政策声明中的目的合理性进行分类。其次，MiniEval融合混合分析与LLM技术，检测第3.1节详述的间接与直接违规行为。此外，该框架引入定量风险评估模块，可对MiniApp生态系统中的隐私风险进行全面量化评估。

2. 背景

本节概述MiniApp架构、用户隐私数据类型及隐私政策。

2.1. MiniApp 基本架构

- **源代码构成：**小程序源代码以结构化格式分发。
例如微信小程序

包含几种关键文件类型：用于配置元数据的 *.json* 文件、用于业务逻辑和 API 访问的 *.js* 文件、用于定义用户界面 (UI) 的 *.wxml* 文件，以及用于为该界面设置样式的 *.wxss* 文件。

- **架构与交互机制：**如图1所示，小程序运行时架构由视图层和逻辑层两大核心组件构成。视图层负责渲染用户界面，逻辑层则处理数据操作、事件响应及 API 调用。两层通过JSBridge机制实现通信，该机制支撑着框架的事件驱动交互模型。该双向通信通道的工作机制如下：当逻辑层更新数据时，调用 *setData()* 方法，JSBridge将此变更传递至视图层以更新UI。反之，视图层中的用户交互（如输入操作）会被封装为事件对象，通过JSBridge传递至逻辑层。这种循环交互机制确保了视图层与逻辑层之间的实时同步。
- **路由与子包机制：**小程序采用基于页面的路由架构。每个页面都封装了特定的逻辑和呈现功能，页面通过路由机制相互连接以支持导航。此外，小程序实现了子包加载策略[9]，该策略可提升性能并缩短初始加载时间。小程序启动时仅加载主包；对应各页面的子包将在访问时动态加载。此设计支持轻量级运行，提升整体用户体验。

2.2. 小程序中的隐私数据与隐私政策

2.2.1. 小程序中的隐私数据

在小程序中，隐私数据的收集主要通过两个渠道：API调用和用户输入。其中，张等学者[6]根据收集的用户数据类型，将小程序的隐私相关API划分为20类。这些API涵盖了从基础个人信息到更敏感数据的广泛隐私数据。例如getLocation API可获取用户的精确地理位置。值得注意的是，此类API采用异步执行模式，其回调函数

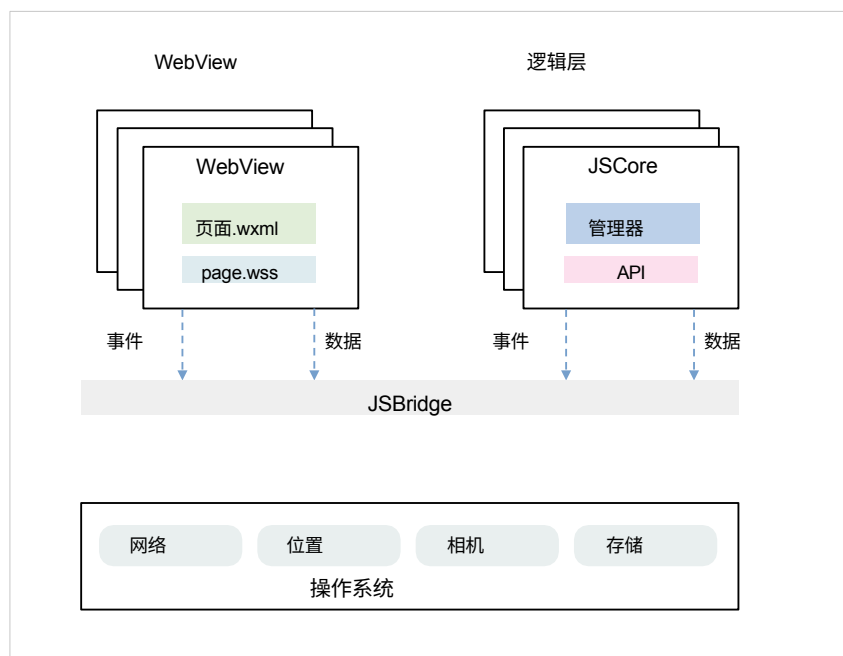


图1：小程序的架构。

成功和失败处理用户授权的结果——分别表示授予或拒绝。该机制引入了隐式控制依赖关系，因为用户的授权决策直接决定了程序后续的执行路径。除了API调用外，用户在使用MiniApp时的输入也可能包含隐私数据。例如，当用户填写表单或使用搜索框时，可能会输入各类个人数据，包括姓名、地址、电话号码等。在研究[6]中，我们识别出17类涉及隐私的用户输入。进一步地，我们根据敏感度对所有隐私数据项进行分级。表1展示了该分类体系，并通过典型示例说明不同类型数据泄露可能造成的危害程度差异。完整的隐私敏感度分类表（含所有数据项的完整映射关系）详见我们的在线文档[10]。

2.2.2. 小程序中的隐私政策

小程序开发者在处理用户数据时，必须填写并遵守相关隐私政策及用户协议。在用户

表1：隐私数据敏感度等级的代表性示例。

级别	敏感性	示例
1	低	网络状态、设备标识符
2	中等	用户昵称、头像
3	高	用户帖子中的上传照片、交易记录
4	关键	精确地理位置、国民身份证号码

开始使用该服务时，开发者必须明确展示其隐私政策并获得知情同意。这些政策应清晰说明将收集的数据类型及其用途。然而在实践中，许多隐私政策未能充分披露所收集的数据或明确界定其使用方式。某些情况下，开发者甚至可能故意隐瞒相关细节，或提供与实际数据处理行为不符的信息。因此，书面政策与实际程序行为之间往往存在显著差异。

3. 动机案例

3.1. 动机案例：小程序中对原则的间接违反

现有针对小程序数据保护原则违规检测的研究，主要通过比对声明的隐私政策与实际收集行为来识别。此方法虽能发现明显违规，却常遗漏隐蔽违规。因此我们区分*直接违规*与*间接违规*。

直接违规行为。我们系统性地将直接违规行为归类为以下三类：

- *遗漏披露*：收集隐私政策未声明的隐私数据，构成对透明原则的根本性违背。
- *冗余披露*：当隐私政策声明收集实际从未收集的隐私数据时，即构成此类违规。此行为通过提供不准确信息违反透明度原则。
- *场景不一致*：当数据收集的实际场景偏离声明的用途时，即构成此类违规。例如，某小程序

可能声明仅在签到时使用位置信息，却在用户个人资料页再次请求该权限。此类矛盾将数据收集与未声明场景挂钩，违反了用途限制原则；同时颠覆用户合理预期，损害知情同意的基础，有损透明度原则。

间接违规。与场景不一致不同，间接违规属于更隐蔽的合规问题。在间接违规案例中，小程序通过限制特定功能访问权限，将功能可用性与用户披露无关数据挂钩。此类限制并非偶然结果，而是程序控制逻辑中刻意设计的强制措施。具体而言，该逻辑将导航、界面渲染等关键操作的执行直接与用户披露隐私数据绑定。

案例1——与隐私相关API绑定功能限制。如图2所示，某购物小程序限制用户访问商品详情页和会员权益信息，除非用户授权位置权限。此案例中，权限请求与功能限制被刻意分离，二者通过hasLocation状态变量及相关控制逻辑实现关联。流程始于用户触发 setDeliveryAddress 函数设置地址。在此合理场景下，调用 wx.getLocation 具有合理性。成功后，其回调函数将 hasLocation 标志设为 true（第10行）。但用户同样有权在 fail 函数中选择拒绝手动设置地址（第12行）。随后在逻辑层(.js)中，if语句(第16行)通过hasLocation标志控制wx.navigateTo调用(第17行)。同样在视图层(.wxml)中，促销横幅的可见性也通过wx:if指令(第4行)依赖于该标志。本质上，该设计滥用了合理场景（设置地址）获取的权限，将其用于控制完全无关的功能。

控制模式摘要（案例1）

- 隐私来源：

- wx.getLocation API 调用。

- 数据依赖性：

- setData调用将值true赋给hasLocation状态变量，从而在两者之间建立数据依赖关系。
- 小程序的数据绑定机制将页面数据对象中的hasLocation值同步至视图组件中对应的wx:if指令。

- **控制流依赖：**

- API成功回调的执行状态决定setData函数是否被调用。
- 条件变量 this.data.hasLocation 通过 if 语句控制 wx.navigateTo 函数的执行。

- **功能限制：**

- wx.navigateTo API 的调用
- 通过 wx:if 指令实现的 <view> 条件渲染

案例2 — 与敏感用户输入关联的功能限制。如图3所示，本案例演示了由敏感用户输入触发的功能限制。小程序仅在用户提供有效电话号码后才会渲染菜单列表。初始状态下，电话状态变量为空字符串（第3行）。包含菜单的<view>组件通过wx:if="phone"条件渲染（第3行），初始值为false。onPhoneInput事件处理器负责验证用户输入。仅当验证成功时，MiniApp才会通过setData()更新phone状态（第8行），从而满足wx:if条件并立即呈现先前隐藏的菜单。该设计使电话号码的披露成为访问菜单内容的先决条件，尽管此数据对查看菜单本身并非必要。

控制模式总结（案例2）

- **隐私来源：**

- <input> 组件中的敏感用户输入。


```

1: // page_shop.js 中的代码
2: Page({
3:   数据: {
4:     hasLocation: false
5:   },
6:
7:   setDeliveryAddress() {
8:     wx.getLocation({success: (res)
9:       => {
10:        this.setData({ hasLocation: true });
11:        setLocation(res) },
12:        fail: () => {inputAddress();}
13:      });
14:   },
15:   getProductDetail(){
16:     if(this.data.hasLocation){
17:       wx.navigateTo({ url: '/pages/product' }); return;
18:     }
19:     wx.showModal({title: '需要定位'});
20:   }
21: });
22: });

1: <!-- page_shop.wxml 中的代码 -->
2: <button bindtap="setDeliveryAddress">设置地址</button>3: <button
bindtap="getProductDetail">查看商品</button>
4: <view wx:if="{{hasLocation}}">满29美元免运费</view>

```

图2：基于隐私相关API调用的功能限制示例（案例1）。

- **数据依赖性：**

- 输入值作为事件对象e的参数流向onPhoneInput处理器。
- 事件对象中的值被赋值给局部常量（第8行），该常量随后作为参数传递给.test()方法进行验证（第9行）。
- setData 调用将经过验证的用户输入赋值给 phone 状态变量，从而建立数据依赖关系。
- MiniApp的数据绑定机制将phone变量的状态同步到视图组件中的 wx:if指令。

- **控制流依赖：**

- 常规条件表达式通过if语句控制 setData函数是否通过if语句执行。

- **功能限制：**

- 通过 wx:if 指令实现 <view> 的条件渲染

检测间接违规的挑战。 现有分析框架难以检测间接违规，因其未能解决三大根本问题：

1. 隐私源建模不完整。 间接违规可能由多重来源触发：不仅涉及隐私相关API调用，还包括用户输入（如表单）中的敏感数据。仅限于建模API来源的检测器必然会遗漏那些基于敏感用户输入控制功能的违规场景。

2. 忽视控制流依赖关系。 间接违规实例背后的逻辑依赖于与隐私数据相关的控制流依赖关系。例如，当用户拒绝权限请求后，会设置一个布尔标志。该标志随后决定条件语句的结果，从而阻止对功能的访问。纯粹的数据流分析可能观察到标志值的变化，但通常无法识别这种变化的后果

```

1: // menuList.js 中的代码
2: Page({
3:   data: { phone: "" },
67:   onPhoneInput(e) {
8:     const value = e.detail.value.trim();9: if
(/^1[3-9]\d{9}$/.test(value)) {10:
    this.setData({ phone: value });
11:   }
12: }
13: });

1: <!-- MenuList.wxml 中的代码 -->
3:       <view class="menu-list">
4:         <text>奶茶</text>
6:       </view>
7:
8:       <view wx:if="{{!phone}}" class="mask">
10:        <input binding="onPhoneInput"/>
12:     </view>

```

图3：基于敏感用户输入的功能限制激励示例（案例2）。

在程序执行路径上。因此，现有工具无法将用户的隐私决策与后续的功能限制建立关联。

3. 忽略功能限制。在间接违规情况下，危害并非数据泄露，而是功能被拒绝。这种不良后果发生在特定程序点，这些点会条件性地阻止用户访问。某些情况下，框架通过限制对`wx.navigateTo`等API的调用来限制导航。在其他场景中，框架通过调用`setData`更新绑定至条件渲染指令（如`wx:if`）的状态变量来限制可见性[11]。由于这些调用不涉及网络数据传输，专为发现信息泄露漏洞设计的工具会完全忽略此类情况。它们未能认识到：限制功能是迫使数据披露的有效手段，且往往更为隐蔽。

我们列举了三起违反数据保护原则的非理性目的真实案例。

案例A——目的异常。声明：位置信息用于登录时的身份验证。

分析：该声明目的与行业惯例不符。

分析：由于位置数据既非唯一性特征又具有时效性，本质上不适用于身份验证。若强制用户披露位置信息完成登录，此举可能导致敏感数据的非必要收集，违反数据最小化原则。

*案例B——目的模糊性。*声明：相机用于拍摄照片。

分析：该小程序提供多个使用摄像头的独立功能（如商品评价、AR试穿）。但隐私政策未将数据收集与具体用途关联，导致用户无法事先知晓具体目的。此声明违反透明度原则。

*案例C——可替代冗余。*声明：身份证号码用于账户注册。

分析：收集身份证号码实属多余。应用程序应改用电话号码注册，既能实现相同功能效果，又减少对隐私的侵犯。此做法违反了数据最小化原则的必要性要求。

4. 方法论

4.1. 框架概述

本方法论通过MiniEval框架实现，其高阶架构如图4所示。该框架由两大核心组件构成：第一组件“违规分析模块”通过行为违规检测与目的合理性分类两大维度识别合规性违规；第二组件“隐私风险评估模块”将识别出的风险量化为单一分数及风险等级。

行为违规检测模块负责基于MiniApp的代码与行为识别原则性违规。该子模块遵循三阶段流程：

1. **MiniApp探索与代码获取：**框架首先通过动态分析系统性探索MiniApp，获取其完整源代码。

2. **通过静态分析构建违规映射：**接下来，静态分析处理获取的代码以识别潜在违规行为。这些结果随后被抽象为两个高级映射：隐私页面映射（M_{pp}）和隐私约束映射（M_{pc}），其定义详见第4.2.2节。

3. **违规验证：**该框架采用混合验证流程。首先通过代码与策略的直接差异匹配检测遗漏披露与冗余披露。对于需要语义理解的复杂场景，则借助大型语言模型（LLM）进行深度验证，具体包含两方面：

- 针对潜在直接违规（源自M_{pp}），执行场景一致性验证以确认实际使用场景是否符合声明场景。
- 针对潜在间接违规（源自M_{pc}），则执行功能必要性评估以判定功能限制是否合理。

第二个模块——目的合理性分类模块，评估数据保护原则的声明维度。该模块运用大型语言模型判断隐私政策中声明的目的是否合理，并将不合理目的分为三类：目的异常、目的模糊及可替代冗余。

最终模块——隐私风险评估模块，基于违规严重性、目的合理性及数据敏感度综合生成隐私风险评分。该评分将映射至对应风险等级，为MiniApp的隐私防护状态提供量化指标。

4.2. 行为违规检测

4.2.1. 应用程序探索与代码获取

我们框架的第一阶段侧重于通过动态探索与运行时监控相结合的方式进行全面数据采集。我们采用基于计算机视觉的爬虫工具动态执行器[12]，自主遍历小程序的用户界面。其主要目标是触发所有子包的动态加载，以便后续进行静态分析。这种探索的完整性至关重要，因为它直接影响召回率

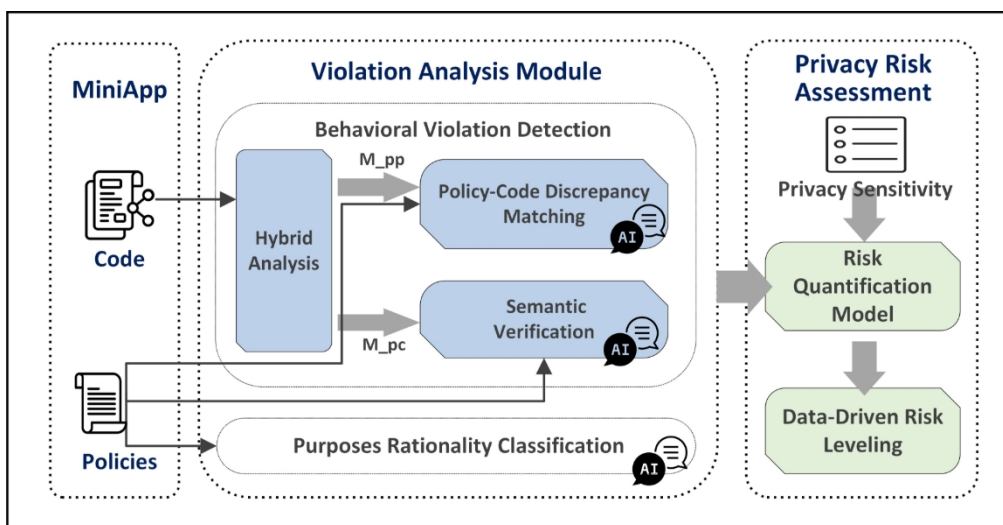


图4: MiniEval整体工作流程

与精确度。任何未探索的页面路径都无法通过静态分析检测，这可能导致遗漏披露的误报——未探索页面上的数据项将无法被识别。反之亦然，这可能引发冗余披露的误报：当爬虫未访问相关页面时，声明的数据项会被错误标记为未使用。

随后，我们使用Unveiler[13]解包获取的软件包，从而获得静态分析所需的完整且结构化的源代码。Unveiler的可靠性确保我们拥有结构良好且可分析的代码库进行静态分析。

同时，我们运用Frida[14]监控MiniApp在遍历过程中的行为。通过钩入敏感API调用，Frida生成动态调用日志。该日志有助于验证静态分析的精准度——例如当静态分析发现冗余披露实例，但动态日志中未出现对应隐私API调用时，可能表明该结果是死代码导致的误报。

4.2.2. 通过静态分析构建违规映射

我们的静态分析构建了支撑违规检测的两个结构化成果：隐私页面映射（ M_{pp} ）和隐私约束映射（ M_{pc} ）。

映射 (M_{pc})。形式上,

$$M_{pp} : \text{隐私数据} \rightarrow \text{页面} \quad (1)$$

$$M_{pc} : \text{隐私数据} \rightarrow \text{受限组件 或 页面} \quad (2)$$

对于 M_{pp} , 每个隐私数据项都关联到使用它的迷你应用页面。对于 M_{pc} , 每个隐私数据项都关联到其访问受该数据限制的 UI 组件或页面。

构建隐私页面映射。 对于 M_{pp} , 我们直接采用王等[15]提出的方法, 该方法构建了"言行一致"框架用于检测小程序代码与隐私政策的一致性。其方法将隐私相关API和用户输入均视为数据源, 通过构建数据依赖图追踪数据在程序中的传播路径。通过将每项数据实践与对应的page.js和page.wxml文件关联, 他们构建了从隐私数据项到使用该数据的小程序页面的映射关系。我们基于其成果构建 M_{pp} 。

构建隐私约束映射。 为系统性构建 M_{pc} , 我们的方法 (i) 识别隐私数据的总体集与潜在功能限制, (ii) 建模控制依赖关系与数据依赖关系, (iii) 应用控制感知隐私流分析 (CAPFA) 提取依赖路径并据此构建 M_{pc} 。

识别隐私数据与潜在功能限制。 在运行CAPFA前, 我们需确定反向遍历的分析终点: 功能限制作为遍历的入口点, 隐私数据则作为其终止点。

- **识别隐私数据。** 我们的隐私数据模型全面涵盖两大主要来源的数据: 隐私相关API调用和敏感用户输入。我们通过特定方法识别每个来源: API调用通过遍历抽象语法树 (AST) 直接识别; 对于用户输入, 则通过正则表达式匹配隐私相关关键词来识别敏感数据。
- **识别潜在功能限制。** 针对导航API (如wx.navigateTo) 的限制, 我们在调用过程中识别其调用行为。

AST遍历并标记为潜在功能限制。对于由`setData`调用强制执行的限制，我们采用两阶段方法：首先使用正则表达式扫描`wxml`文件，提取与条件化UI渲染相关的状态变量（例如受`wx:if`控制的变量）。其次，通过遍历对应的`js`文件抽象语法树进行分析。此阶段识别出修改已提取变量的`setData`调用。通过结合这两步操作，我们能够精确识别条件渲染UI组件中的潜在功能限制。

建模数据依赖与控制依赖。我们利用MiniTracker的赋值流图（AFG）[16]构建数据依赖关系。该模型将程序表示为图结构，其中节点代表程序状态，边代表基于赋值的数据依赖关系。但对MiniTracker实现的审查表明，其对控制依赖的处理仅是部分实现。

对于列表1中的代码，MiniTracker能识别条件到`foo`的依赖关系，却遗漏了条件到`bar`的依赖，因为它仅处理相邻程序状态间的控制依赖。

```
1  if ( condition == true) {  
2    foo = 1; // 语句 1  
3    bar = 2; // 语句 2  
4 }
```

列表 1：控制依赖覆盖示例

为解决此问题，我们遍历抽象语法树（AST）以构建完整的控制依赖关系。在此过程中，条件谓词（如`if`条件）与其主体作用域内的所有程序状态相连。同样，隐私相关的API调用与其成功和失败回调函数作用域内的所有程序状态相连。这两种情况下，我们都在作用域层级构建依赖边。该方法确保每个程序状态都与控制其行为谓词或API调用直接关联。通过这些全面的控制依赖关系纳入赋值流图（AFG），我们构建出控制增强型赋值流图（CEAFG）。我们将其形式化定义为元组 $G = (V, E(d), E(c))$ ，其中：

- V 是一组表示程序状态的节点。

- E_d 是一组数据依赖边 (u, v) ，表示数据从节点 u 流向节点 v 的过程（例如通过赋值操作）。
- E_c 是控制依赖边集 (u, v) ，表示节点 v 的执行受节点 u 控制（例如 if 条件）。

控制感知隐私流分析算法的实现。CAPFA通过遍历CEAFG图结构来识别间接违规路径，该图结构统一了数据与控制依赖关系。如算法1所示，我们的分析定义了两个关键端点。功能限制作为遍历汇点，隐私数据则充当源点。算法随后从这些汇点向源点执行逆向遍历。遍历过程中，算法维护着待访问节点的工作列表、已访问节点的完成集以及依赖关系集。遍历操作流程如下：工作列表初始化为所有汇聚节点（第3行）。算法随后从工作列表迭代处理节点直至清空（第4行）。对于每个待处理节点，算法检索其所有控制或数据依赖关系；记录每条依赖边，并将对应的前驱节点加入工作列表以供后续迭代处理（第7-18行）。遍历终止后（即工作列表清空），基于深度优先搜索的 `generate_paths` 函数将根据收集的依赖边生成所有从汇点返回源点的路径（第20行）。随后通过将每条依赖路径映射回其对应的汇点——即具体函数限制项——构建 M_{pc} 。 M_{pc} 中记录的具体上下文信息取决于限制项的类型：

- 对于导航API，我们首先应用别名分析来解析其url属性。随后获取解析后页面URL的.wxml源代码，并将该代码记录为限制条件的表示形式。
- 对于UI组件，我们记录受限组件的实现上下文。这包括其.wxml代码片段、关联的JavaScript事件处理程序，以及包含该组件的页面.wxml代码。

4.2.3. 直接违规检测

我们的框架系统性地识别三类直接违规：冗余披露、遗漏披露和场景不一致。

算法1 控制感知隐私流分析

```
1: 输入: ceafg, sources, sinks
2: 输出: iv_paths
3: 工作列表  $\leftarrow$  接收端, 已完成  $\leftarrow \emptyset$ , 依赖项  $\leftarrow \emptyset$ 
4: 只要 工作列表  $\neq \emptyset$  则
5:     当前节点  $\leftarrow$  工作列表.pop()
6:     完成  $\leftarrow$  完成  $\cup$  {当前节点}
7:     for 每个谓词  $\in$  get_predecessors(ceafg, current_node) 执行
8:         如果 谓词  $\notin$  done 则
9:             如果 is_control_dep(pred, current_node) 成立, 则
10:                 工作列表  $\leftarrow$  工作列表  $\cup$  {谓词}
11:                 dep  $\leftarrow$  dep  $\cup$  {(当前节点, 谓词)}
12:             end if
13:             if is_assignment(pred, current_node) then
14:                 工作列表  $\leftarrow$  工作列表  $\cup$  {谓词}
15:                 dep  $\leftarrow$  dep  $\cup$  {(当前节点, 谓词)}
16:             结束如果
17:         结束条件语句
18:     结束循环
19: 结束循环
20: iv_paths  $\leftarrow$  generate_paths(dep, sinks, sources)
```

检测流程融合了直接差异比对与基于大型语言模型的深度分析。整个阶段由M_pp规范引导, 该规范作为提取待比对代码与政策声明的清单。具体而言, 差异检查足以识别数据实践 (遗漏披露与冗余披露) 的存在与否, 而检测场景不一致性这一更精细的任务则需要大型语言模型提供的语境理解能力。

*政策代码差异匹配。*通过直接比对申报数据项与我方M_pp中归档的采集项, 可检测遗漏披露与冗余披露。此比对可识别两类违规情形: 当申报项目未出现在M_pp中时, 即构成冗余披露; 当M_pp中存在的项目未在政策中体现时, 则构成遗漏披露。为确保M_pp的精准性 (尤其针对用户输入数据), 我们采用基于大型语言模型的

验证步骤。虽然我们的静态分析使用正则表达式识别所有潜在的输入相关隐私数据，但这可能引入语义误报（例如匹配发生在静态文本标签而非输入字段中）。因此在进行比对前，我们使用大型语言模型（LLM）分析每个正则表达式识别候选项的代码上下文。该模型通过验证元素是否属于数据采集设计的交互组件，有效过滤误报并确保检测的高准确性。

*通过语义分析验证场景一致性。*场景不一致性检测旨在判断小程序的实际使用场景是否与其声明的场景相符。实现这一目标需要对页面代码进行深度语义理解，为此我们运用了大型语言模型（LLM）。在M_pp映射的引导下，LLM对每个页面执行两步评估：(i) 使用场景识别。此步骤中，LLM分析页面的.wxml源代码以识别实际使用场景。
(ii) 一致性验证。该步骤中，LLM将识别出的场景与声明场景进行比对，判断是否一致。

为有效引导LLM进行检测，我们的提示设计融合了多项技术：

- **角色扮演与任务明确定义：**第一步要求LLM从代码推断页面使用场景；第二步则要求其扮演专业隐私审计员角色，并明确任务目标：判定页面实际使用场景是否符合声明内容。
- **上下文锚定：**提示符以结构化格式为大型语言模型提供所有必要上下文：对应的声明以及代表页面实际使用场景的.wxml代码。
- **结构化输出：**要求LLM以严格的JSON格式输出结论（{"is_consistent": boolean, "reason": "..."}），便于程序化解析。

具体而言，我们检测场景不一致性的方法在可扩展性和完整性方面具有显著优势。尽管王世韬等[8]等先前的研究也利用LLM理解场景，但其依赖动态探索和光学字符识别技术。

截图识别（OCR）速度较慢，且可能无法实现全页面覆盖。相比之下，我们的方法通过静态分析直接获取完整源代码。这为大型语言模型提供了整个用户界面的结构化表示，确保更高效且稳健的一致性检查。

4.2.4. 间接违规检测

间接违规检测旨在判定与隐私项目关联的功能限制是否具有功能合理性。该判断需要对受限功能进行深度语义理解，为此我们采用大型语言模型（LLM）完成此任务。针对每项潜在违规，LLM基于M_pc中记录的相关源代码执行两步评估：(i) 功能概括。此步骤中，LLM分析受限元素的相关源代码，生成其实际功能的摘要。(ii) 必要性分析。此步骤中，LLM判定该隐私项目是否为摘要功能所必需。

为有效引导LLM进行检测，我们的提示设计融合了多项技术：

- **角色扮演与明确任务定义：**第一步中，我们指示大型语言模型扮演代码摘要器角色；第二步中，则要求其扮演隐私审计员角色以评估必要性。
- **基于示例的上下文学习：**关键在于，针对必要性评估，我们向LLM提供了间接违规的正式定义，以及一系列合理与不合理的少样本示例（源自第3.1节）。
- **结构化输出：**LLM必须以严格的JSON格式返回判定结果（`{"is_reasonable": 布尔值, "reason": "..."}` ）。

这种基于LLM的功能必要性分析，使我们的框架能够系统性地识别用户被微妙胁迫提供非必要数据的案例。

4.3. 目的合理性分类

目的合理性分类模块对隐私政策进行语义审计。其目标是评估数据收集声明目的是否合理且符合数据保护原则。具体而言，该模块识别三类先前在

：目的异常、目的模糊性及可替代冗余。该分类由大型语言模型（LLM）在精心设计的提示词引导下完成。针对每个声明目的，提示词向LLM提供明确任务定义、三类非理性现象的正式定义，以及各分类的少样本示例（详见第??节）。随后要求LLM以结构化JSON格式输出结果，格式如下：

```
{
  "is_reasonable": true | false,
  "violation_type": "目的异常" | "目的模糊" | "可替代冗余" | "无",
  "reason": "..."
}
```

通过这种基于法律硕士（LLM）的驱动方法，我们的框架能够对隐私政策进行语义审计，隐私政策的语义审计，审视声明用途的有效性。

4.4. 隐私风险评估

作为框架的最终模块，隐私风险评估模块整合前期分析阶段的输出结果，计算出隐私风险评分及风险等级。

4.4.1. 隐私数据的多层次敏感性分类体系

我们风险评估的核心是多级敏感性分类标准。为确保客观严谨性，该分类遵循现行法律法规，包括《中华人民共和国个人信息保护法》(PIPL)[3]及GB/T 45574-2025[17]等国家标准。基于法律框架与实践经验，我们提出一套实用的四级分层体系来评估隐私数据的敏感性。具体分类及其对应的敏感权重 $W_{(sens)(p)}$ 定义如下：

- **第4级：关键敏感性：**此级数据泄露可能危及人身财产安全。示例：*精准地理位置、联系人列表、身份证号码、生物特征数据、财务及健康记录。*

- **三级：高度敏感：**此类数据虽敏感度高，但需结合更多背景信息方可能被严重滥用。通常包含可构建详细用户画像或暴露敏感个人行为及社交关系的数据。示例：用户生成内容（如帖文、媒体文件）及交易记录。
- **二级：中等敏感度：**此级别包含有助于构建用户在线身份或社交档案的数据，但不会直接暴露其现实身份。示例：用户昵称、头像和性别。
- **一级：低敏感度：**此级别包含技术或设备相关数据，直接隐私风险极低。示例：网络状态及特定设备标识符（如IMEI、MAC地址）。

本标准加权风险评分模型提供正式依据。所有分析隐私项与这些级别的完整映射关系详见在线文档[10]。

4.4.2. 风险评分公式与参数定义

基于此敏感性分类体系，我们构建了隐私风险评分——一项量化指标，旨在衡量小程序隐私风险的绝对程度。我们刻意将隐私风险评分设计为累积分数而非百分比指标。百分比指标即使经过加权处理，仍可能产生根本性误导，因为它掩盖了潜在的风险量级。例如，假设两个小程序在两项数据处理实践中各存在一项违规。若小程序A涉及两项低敏感度数据，而小程序B涉及两项极高敏感度数据，两者将呈现相同的50%不合规率。这种误导性等价性未能体现小程序B的更高风险。我们的累积方法可规避此缺陷。隐私风险评分通过以下加权线性模型正式计算：

$$\text{隐私风险评分} = \sum_{p \in P_{app}} W_{sens}(p) \times (\gamma + \alpha \cdot V(p) + \beta \cdot R(p)) \quad (3)$$

其中：

- P_{app} 是与 MiniApp 关联的所有唯一隐私项的集合。

- $W_{sens}(p)$ 是分配给特定隐私项目 p 的敏感度权重。
- $V(p)$ 是二进制标志，指示是否检测到与 p 相关的行为违规。
- $R(p)$ 是二进制标志位，用于标记项目 p 的声明目的是否不合理。
- γ, α, β 分别是表示固有风险权重、行为违规风险权重和合理性风险权重的超参数。

该公式化表达体现了我们的风险评估理念。*超参数* γ 为任何数据收集行为设定非零基准风险，承认即使合规的数据处理也会扩大用户的攻击面。通过赋予 α 和 β 不同的权重，模型能够区分具体行为违规与隐私政策声明缺陷的严重程度，从而生成细致入微且多维度的隐私风险评分。

4.4.3. 数据驱动的风险分级

尽管风险评分提供精细的连续量化结果，但在政策制定和用户沟通中的实际应用需映射为离散可解读的风险等级（如低/高）。核心挑战在于客观界定各等级边界。人工操作不可行，因其本质上具有任意性和主观性，缺乏统计有效性与可重复性。

为客观界定这些边界，我们采用基于数据驱动的聚类方法。其核心原理在于隐私风险评分的统计分布并非均匀，而是蕴含内在结构。聚类算法正是为揭示此类结构而设计，其运作机制是通过划分数据集来最大化群内相似性（凝聚性），同时最小化群间相似性（分离性）。由此形成的边界可视为分布中的自然“断点”，标志着潜在风险特征的显著转变。该方法确保分类结果客观可靠，反映数据内在结构而非主观判断。本方法的实证验证及最优聚类算法的选择详见评估部分（第5.3节）。

5. 评估

本节通过系列实验全面评估MiniEval框架，旨在解答三大核心研究问题：

- **RQ1：合规违规检测有效性。** MiniEval在行为维度和声明维度中检测数据保护原则违规的有效性如何？
- **RQ2：定量风险评估的合理性。** MiniEval隐私风险评估模块背后的方法论具有何种合理性？
- **RQ3：现实世界中的隐私现状。** MiniApp生态系统中的隐私风险现状如何？

5.1. 实验设置

数据集。 本研究采用杨等[18]收集的15,917个微信小程序大规模数据集。该数据集为我们针对小程序生态系统隐私风险的大规模研究（RQ3）提供了基础。我们发现12,365个小程序（77.7%）要么未提供隐私政策，要么声明不收集隐私数据。这一发现揭示了系统性透明度缺失问题，促使我们对各类隐私违规行为展开全面审计。

真实数据。 为建立评估MiniEval有效性（研究问题1）的可靠真实数据，两位精通MiniApp隐私分析的研究人员独立审查了随机抽取的350个MiniApp子集。他们分析了每个应用的源代码、用户界面及隐私政策，以识别所有直接违规、间接违规及非合理用途的实例。

实现与环境。 MiniEval的混合分析引擎采用JavaScript和Python实现。其基于大型语言模型的验证模块使用qwen-plus-2025-07-14模型，调用时设置温度参数为0.2确保输出稳定性。所有实验均在搭载Ubuntu 22.04系统、英特尔酷睿i5-8300H处理器（2.30GHz）及16GB内存的机器上完成。

5.2. 研究问题1：违规检测的有效性

为解答研究问题1，我们基于人工标注的基准数据集评估MiniEval的有效性。评估设计旨在突出MiniEval相较于MiniS-cope[7]等先前的研究引入的新颖分析维度，分为三个阶段：首先，评估其检测直接违规的表现，特别是分析用户输入来源和识别场景不一致的能力；其次，量化其检测间接违规的有效性；最后，评估其识别非理性目的的独特能力。

5.2.1. 检测遗漏与冗余披露的有效性

为评估MiniEval在检测遗漏披露和冗余披露方面的有效性，我们针对个人数据的两大主要来源——隐私相关API和敏感用户输入——对其性能进行了评估。针对API引发的违规行为，我们将MiniEval与尖端工具MiniScope[7]进行基准对比。如表2所示，MiniScope在遗漏披露和冗余披露检测中分别取得97.6%和99.3%的卓越F1分数。MiniEval虽略逊一筹但仍具竞争力，其漏报披露F1值为95.0%，冗余披露F1值达94.5%。这源于MiniEval优先扩展分析范围以涵盖用户输入，但尚未整合可达性分析，导致不可达代码引发误报与漏报。

对于源自用户输入的违规行为，MiniEval采用基于正则表达式的方案生成初始候选项。如表2所示，该方案对用户输入遗漏披露的召回率达85.8%。这些漏检现象源于模式匹配方法的固有局限——既无法覆盖所有可能的关键词，也无法处理图像等非文本内容。同时，该方法对纯模式匹配的依赖必然产生噪声。由于无法理解语义上下文，共产生176个误报，导致精确率仅为69.3%。为解决此问题，我们引入基于大型语言模型的语义过滤器，效果显著。针对用户输入的遗漏披露，LLM通过消除149个误报将精确率从69.3%大幅提升至93.5%。这一显著提升伴随轻微代价：召回率从85.8%微降至83.4%，因LLM过滤器误排了少量真实阳性结果。

表2：基于真实数据的遗漏披露（OD）与冗余披露（RD）检测效果对比。GT、TP、FP、FN分别代表真实数据、真实阳性、假阳性、假阴性。Pre%、Rec%和F1%分别代表精确率、召回率和F1分数。

类别 OD (API)	工具	GT	真阳 性	FP	FN	命中率	Rec%	F1%
OD（输入）	MiniScope [7]	579	571	21	8	96.5%	98.6%	97.6%
	MiniEval-LLM	579	571	52	8	91.7%	98.6%	95.0%
	MiniEval-正则表达式	464	398	176	66	69.3%	85.8%	76.7%
RD（API）	MiniEval-LLM	464	389	27	75	93.5%	83.4%	88.4%
OD（总计）	MiniEval-LLM	1043	960	79	83	92.5%	93.0%	92.7%
RD（输入）	MiniScope [7]	202	199	0	3	100%	98.5%	99.3%
	MiniEval-LLM	202	181	0	21	100%	89.6%	94.5%
	MiniEval-正则表达式	34	30	4	4	88.2%	88.2%	88.2%
	MiniEval-LLM	34	32	5	2	86.5%	94.1%	90.1%
RD（总计）	MiniEval-LLM	236	213	5	23	97.7%	90.3%	93.8%

5.4.2 检测场景不一致性的有效性

为评估MiniEval在检测场景不一致性方面的有效性，我们基于真实数据集对场景不一致性检测模块进行了评估。具体结果详见表3。该模块实现了88.9%的F1分数，其精度为92.3%，召回率为85.7%。虽然这些结果验证了该方法的高精度，但其整体有效性受到较低召回率的限制。召回率受限的根本原因在于Uni-app[19]等第三方框架的模块打包过程。这些框架用专有模块加载器替代直接调用，该加载器通过ID识别模块。这些ID由Webpack[20]等打包工具生成，作为方法的唯一标识符。我们的静态分析器虽能解析原生小程序代码，却无法解析此机制。它无法将模块ID映射回对应的被调用函数，因此无法重建完整的调用链。例如当页面调用外部文件（util.js）中的函数以触发隐私相关API时，我们的工具仅能追踪到加载器的调用，却无法追溯至API调用。这种关联缺失导致无法将隐私数据与使用该数据的页面关联，从而遗漏潜在违规行为的检测，可能降低召回率。

表3：MiniEval场景不一致检测模块在真实数据集上的有效性。GT、TP、FP和FN分别代表真实数据、真阳性、假阳性和假阴性。Pre%、Rec%和F1%分别代表精确率、召回率和F1分数。

违规类型	GT	真阳性	FP	FN	命中率	Rec%	F1%	
情景不一致性		14	12	1	2	92.3%	85.7%	88.9%

5.2.3. 间接违规行为的检测有效性

为评估MiniEval检测间接违规行为的有效性，我们基于真实数据集和大型真实世界数据集进行了两阶段评估。首先，我们评估了MiniEval在真实数据集上的表现。如表4所示，MiniEval正确识别了其中11个案例，F1分数达81.5%，精确率为91.7%，召回率为73.3%。为评估MiniEval在大规模数据集中的精确度，我们将其应用于大规模数据集。静态分析初步识别出2,291个潜在违规案例。经大型语言模型模块复核后，筛选出519个高可信度的间接违规候选项。为验证候选项的精确度，我们人工抽查80个候选项，确认其中71个为真实阳性。这对应88.8%的精确率，证实了本方法在大规模分析中的可靠性。

与准确率相比，我们的召回率明显偏低。为探究假阴性的根本原因，我们分析了相应的召回错误。对假阴性结果的分析揭示了一个根本性局限：我们的静态分析器虽能解析原生MiniApp编程模型，却无法解读第三方框架（如uni-app[19]）引入的非原生编程模式和数据流机制。该问题主要通过两种方式体现：

- 首先，这些框架采用独特的事件处理和数据绑定抽象机制。基于原生语法设计的分析器无法识别这些非原生模式作为用户输入数据的来源，导致无法检测由非原生输入引发的违规行为，从而产生误报。
- 其次，框架引入了复杂的数据检索模式。它们通常采用集中式状态管理库（如Vuex[21]）

作为独立于单个页面的全局数据存储。在多数情况下，关键数据（如 `wx.navigateTo` 等导航 API 的 `url` 参数）会通过框架专属接口（例如 `mapState()` 或 `this.$store.state`）从该存储中获取。我们的静态分析器无法追踪框架特有的数据流。因此，静态分析无法解析具体的URL值，导致无法检测到潜在的导航相关违规行为，从而产生误报。

表4：MiniEval间接违规检测模块在真实数据集上的有效性。GT、TP、FP和FN分别代表真实数据、真实阳性、假阳性、假阴性。Pre%、Rec%和F1%分别代表精确率、召回率和F1分数。

违规类型	GT	真阳性	FP	FN	命中率	Rec%	F1%
间接违规	15	11	1	4	91.7%	73.3%	81.5%

5.2.4. 检测不合理申报目的的有效性

我们基于地面真实数据评估了目的理性分类模块的有效性。为增强分类稳定性，每个声明目的均向大型语言模型提交三次，最终分类结果由多数投票决定。该模块展现出高整体效能，在三类非理性类型中实现宏观平均F1值达90.5%（详见表5）。其在识别最常见的"目的模糊性"时表现尤为突出，F1值达92.2%。在"目的异常"（F1：87.0%）和"可替代冗余"（F1：92.3%）分类中同样表现稳健。所有类别的高精度率（87.0%至100%）表明该模块的阳性识别具有高度可靠性。

通过分析分类错误，我们发现了两个值得注意的模式。假阴性结果的一个来源在于不同非理性类型之间存在微妙的语义重叠。例如，我们观察到可替代冗余被误判为目的模糊性的案例。这表明模型虽能正确识别非理性声明，但在边界案例中难以进行精细区分——例如区分不必要的声明与模糊的声明。相反地，部分假阳性源于大型语言模型（LLM）过度严苛的判定倾向。在若干案例中，模型将目的声明判定为

模糊，因为它们缺乏人类审查员可能认为可接受的具体程度。这凸显了自动化政策审查的固有挑战：在精确语言需求与政策撰写的实际情况之间取得平衡。尽管存在这些局限性，我们基于LLM的方法仍能有效识别隐私声明的合理性。

表5：目的理性分类的有效性。GT、TP、FP和FN分别代表真实值、真阳性、假阳性和假阴性。Pre%、Rec%和F1%分别代表精确率、召回率和F1分数。

非理性类型	GT	真阳性	FP	FN	预%	Rec%	F1%
		性					
目的模糊性	74	71	9	3	88.8%	95.9%	92.2%
目的异常	23	20	3	3	87.0%	87.0%	87.0%
可替换冗余	7	6	0	1	100%	85.7%	92.3%

5.3. RQ2：定量风险评估的可靠性

本节验证了隐私风险评估模块的合理性。隐私风险评分采用精心配置超参数的公式计算得出。我们赋予行为违规 ($\alpha = 1.0$) 的权重高于非理性声明目的 ($\beta = 0.5$)。该权重方案基于以下原则：具体程序行为比声明性陈述具有更大的内在风险。所有数据收集操作均应用非零基线风险 ($\gamma = 0.1$)，以体现扩大用户攻击面的固有成本。此外，敏感度权重 ($W_{(sens)}$) 采用等比递进方式分配 (1, 2, 4, 8)。此设计选择体现了核心原理：数据潜在风险并非随敏感度线性增长，而是呈指数级攀升。

应用于我们的数据集后，生成的隐私风险评分呈现出右偏分布且具有长尾特征，如图5所示。该分布表明，有相当一部分小程序展现出极高的风险特征。因此在建立分级体系时，我们首先将评分最高的5%数据标记为极端值。随后对剩余95%数据进行比较评估，测试三种分区方法：分位数法、K均值聚类法和詹克斯自然断点聚类法。我们首先评估分位数法但发现其不适用。如图5所示，由于评分在特定数值附近高度集中，该方法将相同评分分配给不同数据点，导致分类一致性失效。

层级。后续分析聚焦于K均值聚类与詹克斯自然断点法，分别采用轮廓系数和方差拟合优度（GVF）评估其表现。如表6所示，两项指标均支持四层分类作为最优方案：GVF在k=4处呈现明显拐点，而K均值聚类的轮廓系数在同值时也保持较高水平。总体而言，两项指标均指向k=4作为最优解，实现了统计拟合度与模型可解释性的平衡。值得注意的是，K均值法与詹克斯自然断点法对四级结构的边界划分完全一致（见表7），有力证明该结构反映了数据的内在特征。基于这些边界，我们将最终风险等级定义为低、中、高、严重四个层次。前三级风险等级遵循聚类边界划分，而"严重"级则将最高风险聚类与前5%的极端评分合并，确保所有极端风险特征均被纳入。

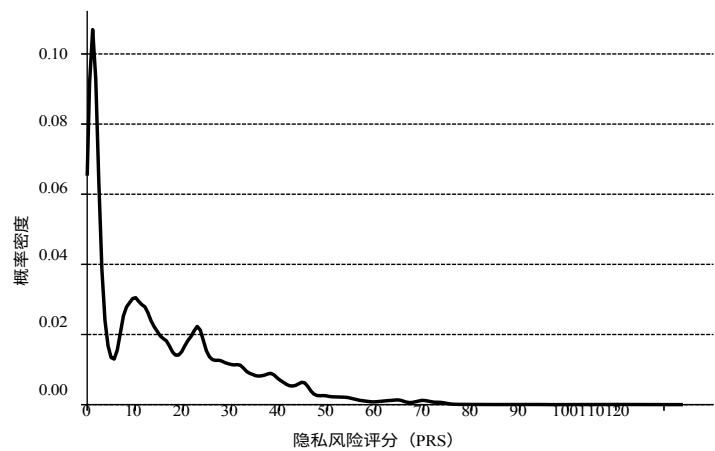


图5：隐私风险评分的核密度估计（KDE）。分布呈现高度右偏态，具有显著的长尾和多重峰值

。

表6：K均值聚类与詹克斯自然断点聚类的聚类验证指标。

层级 (k)	轮廓分数	GVF
2	0.6680	0.7504
3	0.6293	0.8858
4	0.6913	0.9484
5	0.6870	0.9678
6	0.7001	0.9792
7	0.6959	0.9845

表7：最终数据驱动风险等级与评分分界线。请注意，严重等级为复合类别，涵盖前5%的极端评分。

风险等级	分数范围	应用程序数量
1级（低）	0.00 – 6.60	5,749
二级（中等）	6.60 – 17.60	4,395
三级（高）	17.60 – 30.50	3,139
第4级（严重）	30.50+	2,634

5.4. RQ3：现实世界隐私现状

本节对微信小程序生态系统内的隐私实践进行了大规模分析。我们的研究发现，该生态系统存在系统性违规现象，隐私侵犯行为普遍存在。此外，定量评估表明由此产生的隐私风险与应用程序类别高度相关。

*违规现象普遍性*分析表明，小程序生态中的合规违规现象既普遍存在又形式多样，不仅限于代码实践与政策声明的直接矛盾。具体而言，我们发现了807例场景不一致、4929例涉及非合理用途的声明，以及689例间接违规。这些发现凸显了复杂隐蔽的隐私风险普遍存在。

风险态势的定量分析。通过大规模分析发现，36.2%的小程序属于高风险或严重风险等级，表明该问题普遍存在。进一步分析显示，隐私风险与小程序的应用类别存在显著相关性。如表8和图6所示，围绕现实交易的核心类别（如支付、银行、社交、购物

等)存在较高风险。

电子商务与零售、旅游与交通两大领域始终呈现高风险评分（平均隐私风险评分>23.0）。风险升高的根源在于其业务领域特性。用户为获取核心服务必须提供身份信息、联系方式等敏感数据。因此，这两个类别中分别有58.7%和56.7%的小程序被划入最高两级风险等级。这些发现表明，小程序生态系统中的隐私风险并非均匀分布，而是倾向于聚集在那些对用户数据需求更高的应用类别中。

表8：主要小程序类别的关键风险指标。按平均隐私风险评分(Avg)排序。高/严重%代表处于最高两级风险的小程序占比。

类别	数量	平均	高/严重%
房地产	108	23.72	61.11%
医疗与健康	319	23.59	60.81 %
电子商务与零售	1974	23.45	58.66%
旅游与运输	379	23.13	56.73%
企业与专业服务	1711	22.86	57.57 %
政府与公共服务	165	21.67	61.82 %
本地与生活服务	1061	21.28	52.03%
金融与支付	277	20.75	52.35 %
农业	110	19.79	48.18 %
体育	81	18.28	44.44%
工具	306	15.51	39.54%
社交网络	495	14.27	30.71%
教育	1072	12.80	29.29 %
信息与新闻	630	10.51	26.83 %
内容与娱乐	3789	7.97	13.67%

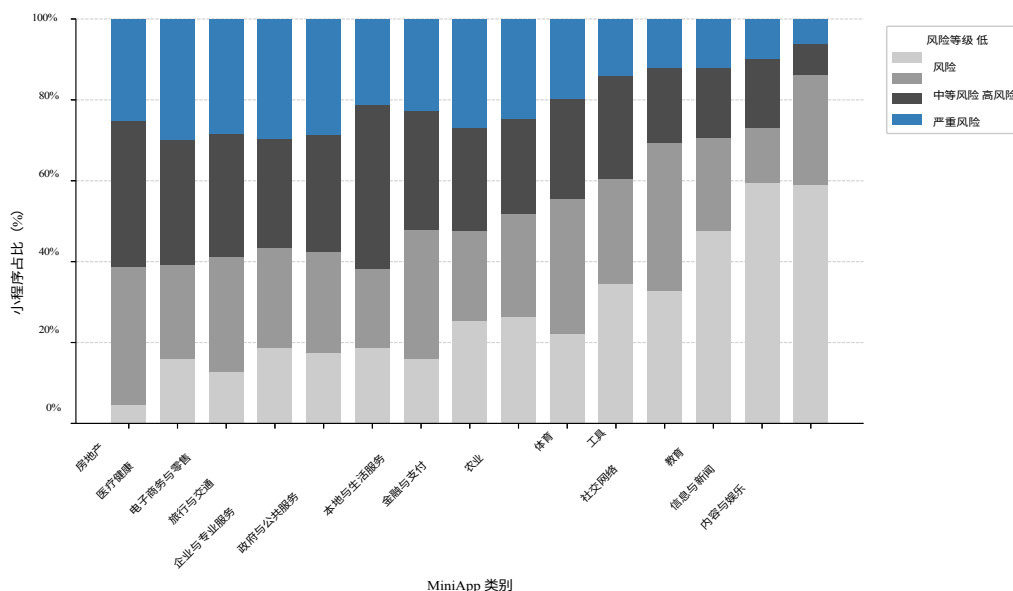


图 6：按平均风险排序的迷你应用类别风险水平分布。

6. 限制

尽管我们的框架展现出高效能，但我们承认其存在若干局限性，这些局限性为未来工作指明了明确方向。首先，风险评分公式可进一步细化。当前模型对行为违规类别的所有违规类型赋予单一统一权重，对非理性目的类别的所有类型也采用统一权重。例如，该模型无法区分开发者疏忽导致的"披露遗漏"与开发者蓄意制造的"间接违规"。未来工作的重点之一是探索机器学习技术，直接从数据中学习每种具体违规类型的精细化风险参数。其次，我们的数据敏感性分类体系虽基于法律框架，但依赖人工分类可能遗漏情境依赖的细微差异。此处同样可通过数据驱动方法构建更稳健的模型。最后，我们的静态分析引擎存在已知局限：解析第三方框架的困难及路径可达性分析的缺失，分别可能导致部分漏报与误报。未来工作还将聚焦于增强

引擎的能力，包括集成可达性分析模块以提升精度，并将整个MiniEval框架扩展至支付宝、TikTok等其他平台以验证其泛化能力。

7. 相关工作

7.1. 微信小程序安全

微信小程序安全研究在范围和深度上不断深化。早期工作侧重于建立基础知识和检测框架。例如，王涛等人[22]对83个微信小程序漏洞进行了首次实证研究，并开发了名为WeDetector的检测工具。在此基础上，韩等学者[23]提出了针对小程序的综合性三元威胁模型，而杨等学者[18]则针对特定攻击向量开发了CmrfScanner工具，用于检测跨小程序请求伪造（CMRF）漏洞。后续研究进一步深入API与权限层，揭示出更隐蔽的实现缺陷。例如，Li等人[24]发现了通过不当文件管理导致敏感数据泄露等关键缺陷；Che等人[25]则系统研究了"API作用域错配"——一种源于API与权限范围映射错误的新型漏洞。除技术漏洞外，研究者还聚焦于权限请求过程中的滥用行为。例如王等人开发了MiniChecker[26]来应对胁迫性与骚扰性权限请求。通过构建通用函数调用图，该工具可自动识别五类滥用模式，包括主页弹窗、重复弹窗及循环弹窗。另一关键研究方向聚焦于追踪敏感数据流以直接检测隐私泄露。例如TaintMini[27]利用静态污点分析检测敏感数据流向，MiniTracker[16]则通过对15万微信小程序的大规模研究揭示了常见隐私泄露模式。此外，其他研究开始考察应用实际数据处理行为与其隐私政策声明的一致性。王寅等[15]在此领域发现，高达89.4%的应用程序违反了自身隐私政策[8]。

7.2. 软件风险量化与评估

Enck等人[28]提出了一种系统，用于识别应用程序是否使用了潜在危险的权限组合。为此，他们通过人工方式

定义了一组代表风险的权限组合。若应用程序通过静态分析使用这些组合，则被标记为恶意应用。H Hao等[29]通过统计分析同类良性应用的权限请求分布，构建了标准权限向量模型。随后通过计算应用权限向量与标准模型间的欧几里得距离，评估Android应用的风险值。HX Son等[30,31]提出基于混合分析的方法。通过静态分析提取应用权限请求与数据收集行为生成应用签名，结合类别签名与语义相似度比较目标应用签名，量化权限请求的偏差程度。同时运用动态行为分析调整权重，生成更精准的风险评估结果。Merlo和Georgiu[32]推出的RiskIn-Droid工具可有效评估Android应用风险。该工具基于四类应用权限定义特征向量，并采用特定分类器实施监督学习。PRADroid框架[33]则通过由决策树等五种分类器组成的集成模型评估风险概率，从应用中提取并编码24项危险权限特征。

8. 结论

MiniEval是首个从两大关键维度系统性审计MiniApp生态的框架。首先，在违规检测方面，它不仅能识别明确的合规性违规行为，还能发现我们新定义的"间接违规"和"非理性目的"概念。其次，在风险评估方面，它将检测到的违规行为转化为量化的隐私风险评分，并运用聚类方法自动将每个MiniApp划分为明确的风险等级。此外，我们在包含350个真实应用的基准数据集上验证了MiniEval的高效性。后续对超过15,000个MiniApp的大规模分析表明，严重隐私风险不仅普遍存在，更在数据密集型应用类别中呈现过度集中态势。我们的研究确立了MiniApp业务领域与其隐私风险之间清晰的数据驱动关联。

声明

本文撰写过程中，作者使用ChatGPT和Gem-ini工具提升可读性。使用工具后，作者已对内容进行必要审阅与编辑，并对发表文章内容承担全部责任。

参考文献

- [1] 欧洲委员会, 《通用数据保护条例》, https://commission.europa.eu/law/law-topic/data-protection_en, 访问日期: 2025年9月8日 (2022年)。
- [2] 加利福尼亚州司法部, 《加州消费者隐私法案》, <https://oag.ca.gov/privacy/ccpa>, 访问日期: 2025年9月8日 (2022年)。
- [3] 个人信息保护委员会, 《个人信息保护法》, <https://www.ppc.go.jp/>, 访问日期: 2025年9月8日 (2022年)。
- [4] 加拿大政府, 《消费者隐私保护法》, <https://ised-isde.canada.ca/site/innovation-better-canada/en/consumer-privacy-protection-act>, 访问日期: 2025年9月8日 (2022年)。
- [5] 腾讯微信团队, 《微信隐私政策》, https://www.wechat.com/en/privacy_policy.html, 访问日期: 2025年9月8日 (2023年)。
- [6] 张晓、王洋、张晓、黄志、张亮、杨明, 《解析微信子应用生态中的隐私过度收集现象》(2023)。arXiv:2306.08391, doi:10.48550/arXiv.2306.08391。URL <https://arxiv.org/abs/2306.08391>
- [7] S. 王, Y. 李, K. 王, Y. 刘, H. 李, Y. 刘, H. 王, Minis-cope: 基于两阶段迭代混合分析的迷你应用自动化界面探索与隐私不一致检测, ACM软件工程方法学汇刊 34 (6) (2025年7月).doi:10.1145/3709351.

- [8] 王思，赵玮，王凯，王浩，基于使用场景的小程序数据最小化研究，收录于：
2023年ACM安全可信超级应用研讨会论文集（SaTS '23），计算机协会

计算机协会, 纽约州纽约市, 2023年, 第29–32页。doi: 10.1145/3605762.3624435.

- [9] W3C, 小程序子打包规范, 访问日期: 2025年9月8日 (2023).
URL <https://www.w3.org/TR/mini-app-white-paper/> #子包装
- [10] 在线文档, 访问日期: 2025年9月8日 (2025年) 。
URL <https://docs.google.com/spreadsheets/d/1BzMkHFyPR5UCWj-eZzuC6IREqJioEhCL/edit?usp=sharing&ouid=100223931989156348767&rtpof=true&sd=true>
- [11] 腾讯微信团队, 《Wxml 语法参考 / 条件渲染》, <https://developers.weixin.qq.com/miniprogram/dev/reference/wxml/conditional.html>, 访问日期: 2025年9月8日。
- [12] S. Li, Z. Yang, Y. Yang, D. Liu, M. Yang, 大规模识别移动小程序中的跨用户隐私泄露, IEEE 信息取证与安全汇刊 19 (2024) 3135–3147. doi:10.1109/TIFS.2024.3356197.
- [13] unweiler, unweiler, [计算机软件] 访问日期: 2025年9月8日。
URL <https://github.com/0day404/unveilr>
- [14] Frida, Frida, <https://github.com/frida/frida>, [计算机软件] 访问日期: 2025年9月8日。
- [15] 王勇, 范明, 刘杰, 陶军, 金伟, 王浩, 熊强, 刘涛, 言行一致: 小程序中程序代码与隐私政策数据实践一致性检测, IEEE 软件工程学报 50 (12) (2024) 3225–3248. doi:10.1109/TSE.2024.3479288.
- [16] W. 李, B. 杨, H. 叶, L. 向, Q. 陶, X. 王, C. 周, 《迷你追踪器: 小型应用程序中的大规模敏感信息追踪》, IEEE 可靠安全计算汇刊 21 (4) (2024) 2099–2114. doi:10.1109/TDSC.2023.3299945.
- [17] 中国国家市场监督管理总局与国家标准化管理委员会, 数据安全技术——敏感个人信息处理安全要求

处理敏感个人信息的安全要求（GB/T 45574-2025），中华人民共和国国家标准，访问日期：2025年9月8日（2025年4月）。

- [18] 杨勇，张勇，林志，跨微应用请求伪造：根源、攻击与漏洞检测，载于：2022年ACM SIGSAC计算机与通信安全会议论文集，CCS '22，美国计算机协会，纽约州纽约市，2022年，第 3079–3092页。 doi:10.1145/3548606.3560597。
- [19] DCloud, Uni-app, 访问日期：2025年9月8日 (2025).
URL <https://uniapp.dcloud.io/>
- [20] Webpack 社区，Webpack，访问日期：2025年9月8日（2025年）。
URL <https://webpack.js.org/>
- [21] Vue.js 团队，Vuex，访问日期：2025年9月8日（2025年）。
URL <https://vuex.vuejs.org/>
- [22] T. Wang, Q. Xu, X. Chang, W. Dou, J. Zhu, J. Xie, Y. Deng,
杨杰，杨杰，魏杰，黄涛，微信小程序中缺陷的特征分析与检测，载于：第44届国际软件工程大会论文集，ICSE '22，美国计算机协会，纽约州纽约市，2022年，第363–375页。doi:10.1145/3510003.3510114.
- [23] 韩宇，季晓，王志，张杰，小程序安全与漏洞系统性分析，载于：《2023年ACM安全可信超级应用研讨会论文集》，SaTS '23，美国计算机协会，纽约州纽约市，2023年，第1-9页。doi:10.1145/3605762.3624432.
- [24] 李杰，张勇，周伟，张思，探索小程序中的权限控制缺陷，收录于：2024年IEEE第23届计算与通信可信、安全与隐私国际会议（Trust-Com），IEEE，2024年，第1935–1941页。doi:10.1109/TrustCom63139.2024.00268.

- [25] J. Che, C. Guo, N. Dong, J. Pei, L. Fan, X. Mi, X. Xie, X. Luo, Z. Liu, R. Cheng, 《揭示应用内生态系统中的API作用域错位》, ACM软件工程会议论文集2(ISSTA) (2025) 1933–1954. doi:<https://doi.org/10.1145/3728962>.
- [26] 王宇, 范明, 周浩, 王浩, 金伟, 李军, 陈伟, 李松, 张宇, 韩东等, Minichacker: 检测小程序中滥用权限请求行为的数据隐私风险, 收录于: 第39届IEEE/ACM国际自动化软件工程会议论文集, 2024年, 第1667–1679页。doi:<https://doi.org/10.1145/3691620.3695534>.
- [27] 王晨, 柯瑞, 张宇, 杨宇, 林志, Taintmini: 基于静态污点分析检测小程序中敏感数据的流动, 载于: 第45届国际软件工程大会论文集, ICSE '23, IEEE出版社, 澳大利亚维多利亚州墨尔本, 2023年, 第932–944页。doi:[10.1109/ICSE48619.2023.00086](https://doi.org/10.1109/ICSE48619.2023.00086).
- [28] W. 恩克, P. 吉尔伯特, S. 韩, V. 滕杜尔卡, B.-G. 闫, L. P. 考克斯, J. Jung, P. McDaniel, A. N. Sheth, Taintdroid: 智能手机实时隐私监控的信息流追踪系统, 《ACM计算机系统汇刊》32(2) (2014) 1–29. doi:<https://doi.org/10.1145/2619091>.
- [29] H. Hao, Z. Li, H. Yu, 《安卓应用风险测量与评估的有效方法》, 载于: 2015年软件工程理论国际研讨会论文集, 2015年, 第31–38页。doi:[10.1109/TASE.2015.16](https://doi.org/10.1109/TASE.2015.16).
- [30] 孙赫翔, 卡米纳蒂·布鲁诺, 费拉里·埃莱奥诺拉, 安卓应用风险评估机制, 载于: 2021年IEEE智能物联网国际会议 (SmartIoT), 2021年, 第237–244页。doi:[10.1109/SmartIoT52359.2021.00044](https://doi.org/10.1109/SmartIoT52359.2021.00044).
- [31] 孙赫旭、卡米纳蒂·B、法拉利·E, 基于混合分析的安卓应用风险评估机制, 《数据科学与工程》第7卷第3期 (2022年), 第242–页。

- [32] A. Merlo, G. C. Georgiu, Riskindroid: 基于机器学习的安卓风险分析, 载于:
《ICT系统安全与隐私保护: 第32届IFIP TC 11国际会议, SEC 2017, 意大利罗马, 2017年5月29-31日》, 会议论文集第32卷, Springer出版社, 2017年, pp. 538–552. doi:https://doi.org/10.1007/978-3-319-58469-0_36.
- [33] 杨勇, 杜晓, 杨志强, Pradroid: 安卓应用隐私风险评估, 收录于: 2021年IEEE第五届密码学、安全与隐私国际会议 (CSP), 2021年, 第 90–95 页. doi: 10.1109/CSP51677.2021.9357608.